Credit Risk Modelling - Loan Classification

By- Elina Kamboj

INTRODUCTION

The Loan Classification project is designed to help one understand and apply machine learning techniques to classify loan applicants based on their likelihood of loan repayment.

PREREQUISITES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import svm
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

import sklearn
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report , accuracy_score , precision_score , confusion_matrix , recall_score , f1_score
from sklearn.svm import SVC
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier,AdaBoostClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import AdaBoostClassifier

import warnings
warnings.filterwarnings("ignore")
```

DATASET OVERVIEW

```
df = pd.read_csv('loan_detection.csv')

print(df.shape)
print(df.head())
print(df.info())
print(df.describe())
```

```
#checking null values
print(df.isnull().sum())
```

```
       age  campaign  pdays  previous  ...  poutcome_failure  poutcome_nonexistent  poutcome_success  Loan_Status_label
0       56         1    999         0  ...                 0                     1                 0                  0
1       57         1    999         0  ...                 0                     1                 0                  0
2       37         1    999         0  ...                 0                     1                 0                  0
3       40         1    999         0  ...                 0                     1                 0                  0
4       56         1    999         0  ...                 0                     1                 0                  0
...    ...       ...    ...       ...  ...               ...                   ...               ...                ...
41183   73         1    999         0  ...                 0                     1                 0                  1
41184   46         1    999         0  ...                 0                     1                 0                  0
41185   56         2    999         0  ...                 0                     1                 0                  0
41186   44         1    999         0  ...                 0                     1                 0                  1
41187   74         3    999         1  ...                 1                     0                 0                  0

[41188 rows x 60 columns]
```

```
                age      campaign         pdays      previous  ...  poutcome_failure  poutcome_nonexistent  poutcome_success  Loan_Status_label
count  41188.00000  41188.000000  41188.000000  41188.000000  ...      41188.000000          41188.000000      41188.000000       41188.000000
mean      40.02406      2.567593    962.475454      0.172963  ...          0.103234              0.863431          0.033335           0.112654
std       10.42125      2.770014    186.910907      0.494901  ...          0.304268              0.343396          0.179512           0.316173
min       17.00000      1.000000      0.000000      0.000000  ...          0.000000              0.000000          0.000000           0.000000
25%       32.00000      1.000000    999.000000      0.000000  ...          0.000000              1.000000          0.000000           0.000000
50%       38.00000      2.000000    999.000000      0.000000  ...          0.000000              1.000000          0.000000           0.000000
75%       47.00000      3.000000    999.000000      0.000000  ...          0.000000              1.000000          0.000000           0.000000
max       98.00000     56.000000    999.000000      7.000000  ...          1.000000              1.000000          1.000000           1.000000

[8 rows x 60 columns]
```

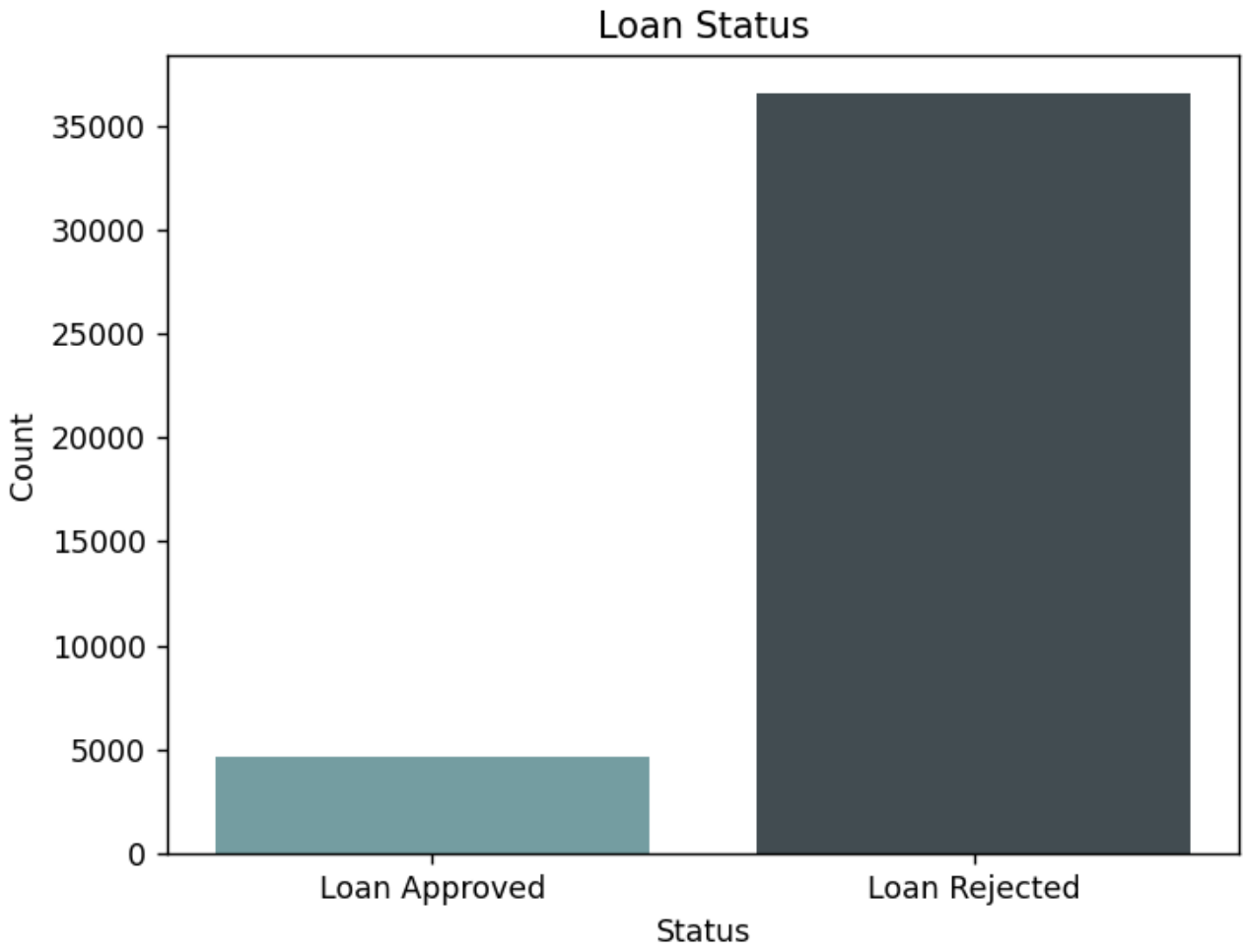
EDA

```
53  #checking the count for loan approved and loan rejected
54  print(df['Loan_Status_label'].value_counts())
55
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Loan_Status_label
0    36548
1     4640
Name: count, dtype: int64
```

```
#visualisation of approved vs rejected loan applications with the help of a barplot
status = list(df.Loan_Status_label)
approved = status.count(1)
rejected = status.count(0)
labels = ['Loan Approved', 'Loan Rejected']
counts = [approved, rejected]
plt.bar(labels, counts, color=['#749da1', '#424c51'])
plt.xlabel('Status')
plt.ylabel('Count')
plt.title('Loan Status')
print(plt.show())
```




```
#correlation between other columns or features and 'Loan_Status_label'
correlation_matrix = df.corr()
print(correlation_matrix['Loan_Status_label'])
```

```
age                             0.030399
campaign                       -0.066357
pdays                          -0.324914
previous                        0.230181
no_previous_contact            -0.324877
not_working                     0.121246
job_admin.                      0.031426
job_blue-collar                -0.074423
job_entrepreneur               -0.016644
job_housemaid                  -0.006505
job_management                 -0.000419
job_retired                     0.092221
job_self-employed              -0.004663
job_services                   -0.032301
job_student                     0.093955
job_technician                 -0.006149
job_unemployed                  0.014752
job_unknown                    -0.000151
marital_divorced               -0.010608
marital_married                -0.043398
marital_single                  0.054133
marital_unknown                 0.005211
education_basic.4y             -0.010798
education_basic.6y             -0.023517
education_basic.9y             -0.045135
education_high.school          -0.007452
education_illiterate            0.007246
education_professional.course   0.001003
education_university.degree     0.050364
education_unknown               0.021430
default_no                      0.099344
default_unknown                -0.099293
default_yes                    -0.003041
housing_no                     -0.011085
housing_unknown                -0.002270
housing_yes                     0.011743
loan_no                         0.005123
loan_unknown                   -0.002270
loan_yes                       -0.004466
contact_cellular                0.144773
contact_telephone              -0.144773
month_apr                       0.076136
month_aug                      -0.008813
month_dec                       0.079303
month_jul                      -0.032230
month_jun                      -0.009182
month_mar                       0.144014
month_may                      -0.108271
month_nov                      -0.011796
month_oct                       0.137366
month_sep                       0.126067
day_of_week_fri                -0.006996
day_of_week_mon                -0.021265
day_of_week_thu                 0.013888
day_of_week_tue                 0.008046
day_of_week_wed                 0.006302
poutcome_failure                0.031799
poutcome_nonexistent           -0.193507
poutcome_success                0.316269
Loan_Status_label               1.000000
Name: Loan_Status_label, dtype: float64
```

SPLITTING DATA INTO TRAIN AND TEST

```
83   #splitting data into train and test sets
84   x = df.drop('Loan_Status_label', axis=1)
85   y = df['Loan_Status_label']
86   x_train, x_test, y_train, y_test= train_test_split(x , y , test_size = 0.2 , random_state = 42)
87
88   print("X_train shape:", x_train.shape)
89   print("X_test shape :", x_test.shape)
90   print("y_train shape:", y_train.shape)
91   print("y_test shape:", y_test.shape)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

X_train shape: (32950, 59)
X_test shape : (8238, 59)
y_train shape: (32950,)
y_test shape: (8238,)
```

```
powershell
powershell
```

---

1. Logistic Regression

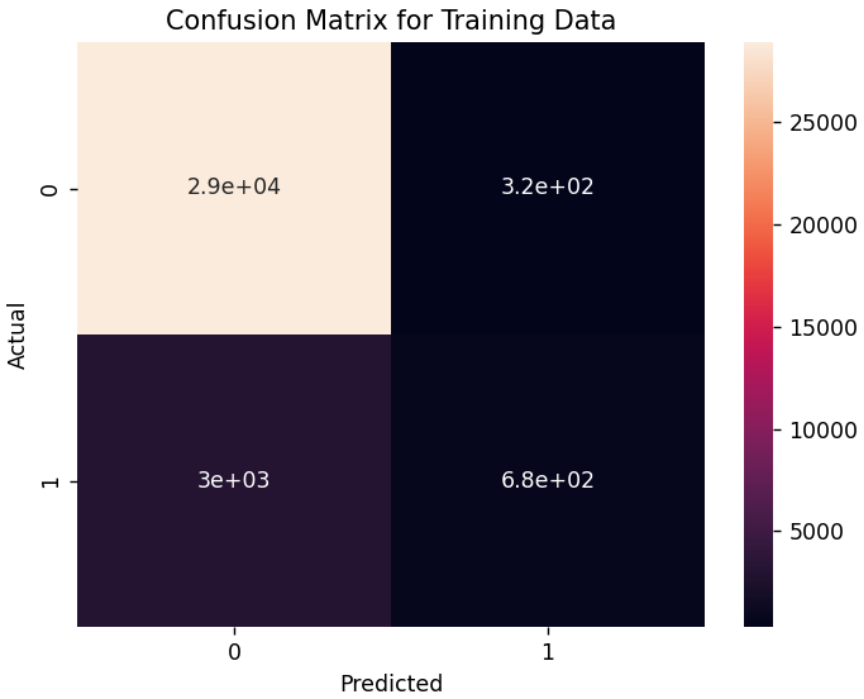
```python
#Model Training and Prediction
lr = LogisticRegression()
lr.fit(x_train , y_train)

y_train_pred = lr.predict(x_train)
y_test_pred= lr.predict(x_test)


#Training Evaluation
print(confusion_matrix(y_train, y_train_pred))
print(accuracy_score(y_train, y_train_pred))
print(classification_report(y_train, y_train_pred))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

powershell
powershell

```
[[28925   320]
 [ 3023   682]]
0.8985432473444613
              precision    recall  f1-score   support

           0       0.91      0.99      0.95     29245
           1       0.68      0.18      0.29      3705

    accuracy                           0.90     32950
   macro avg       0.79      0.59      0.62     32950
weighted avg       0.88      0.90      0.87     32950
```

```python
#Plotting Heatmap for Training Data
sns.heatmap(confusion_matrix(y_train, y_train_pred), annot=True)
plt.title('Confusion Matrix for Training Data')
plt.xlabel('Predicted')
plt.ylabel('Actual')
print(plt.show())
```



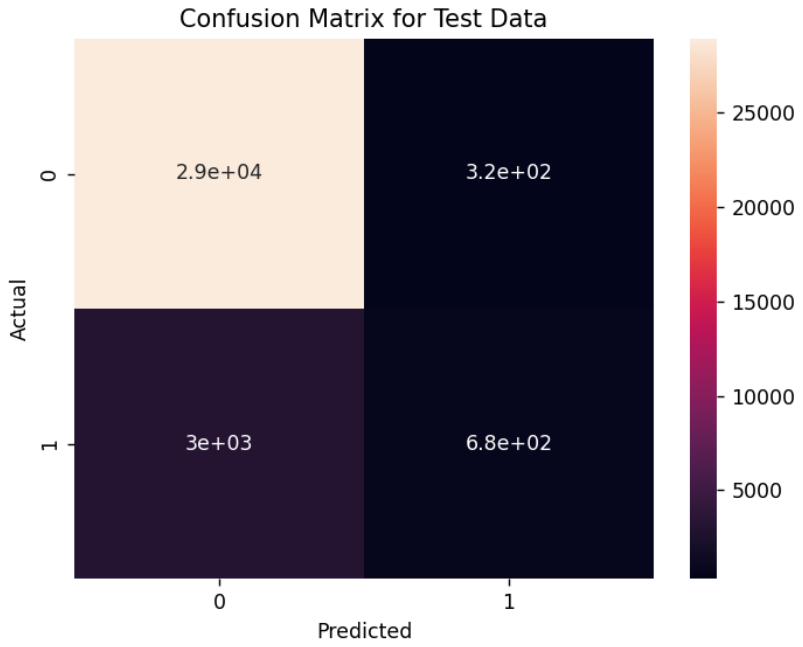

```python
#Test Evaluation
print(confusion_matrix(y_test, y_test_pred))
print(accuracy_score(y_test, y_test_pred))
print(classification_report(y_test, y_test_pred))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

powershell
powershell

```
[[7210   93]
 [ 763  172]]
0.896091284292304
              precision    recall  f1-score   support

           0       0.90      0.99      0.94      7303
           1       0.65      0.18      0.29       935

    accuracy                           0.90      8238
   macro avg       0.78      0.59      0.62      8238
weighted avg       0.88      0.90      0.87      8238
```

```python
#Plotting Heatmap for Test Data
sns.heatmap(confusion_matrix(y_train, y_train_pred), annot=True)
plt.title('Confusion Matrix for Test Data')
plt.xlabel('Predicted')
plt.ylabel('Actual')
print(plt.show())
```



2. SVM Classifier

```python
#SVM Model Building
svm=SVC()
svm.fit(x_train , y_train)
y_train_pred = svm.predict(x_train)
y_test_pred = svm.predict(x_test)

#Training
print(confusion_matrix(y_train, y_train_pred))
print(accuracy_score(y_train, y_train_pred))
print(classification_report(y_train, y_train_pred))

#Testing
print(confusion_matrix(y_test, y_test_pred))
print(accuracy_score(y_test, y_test_pred))
print(classification_report(y_test, y_test_pred))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[[28819   426]
 [ 2928   777]]
0.8982094081942337
              precision    recall  f1-score   support

           0       0.91      0.99      0.95     29245
           1       0.65      0.21      0.32      3705

    accuracy                           0.90     32950
   macro avg       0.78      0.60      0.63     32950
weighted avg       0.88      0.90      0.87     32950

[[7181  122]
 [ 745  190]]
0.8947560087399854
              precision    recall  f1-score   support

           0       0.91      0.98      0.94      7303
           1       0.61      0.20      0.30       935

    accuracy                           0.89      8238
   macro avg       0.76      0.59      0.62      8238
weighted avg       0.87      0.89      0.87      8238
```

3. KNN Classifier

```python
#KNN Model Building
knn= KNeighborsClassifier(n_neighbors = 5)
knn.fit(x_train , y_train)
y_train_pred = knn.predict(x_train)
y_test_pred = knn.predict(x_test)

#Training
print(confusion_matrix(y_train, y_train_pred))
print(accuracy_score(y_train, y_train_pred))
print(classification_report(y_train, y_train_pred))

#Testing
print(confusion_matrix(y_test, y_test_pred))
print(accuracy_score(y_test, y_test_pred))
print(classification_report(y_test, y_test_pred))
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[[28852   393]
 [ 2608  1097]]
0.9089226100151745
              precision    recall  f1-score   support

           0       0.92      0.99      0.95     29245
           1       0.74      0.30      0.42      3705

    accuracy                           0.91     32950
   macro avg       0.83      0.64      0.69     32950
weighted avg       0.90      0.91      0.89     32950

[[7125  178]
 [ 740  195]]
0.8885651857246905
              precision    recall  f1-score   support

           0       0.91      0.98      0.94      7303
           1       0.52      0.21      0.30       935

    accuracy                           0.89      8238
   macro avg       0.71      0.59      0.62      8238
weighted avg       0.86      0.89      0.87      8238
```

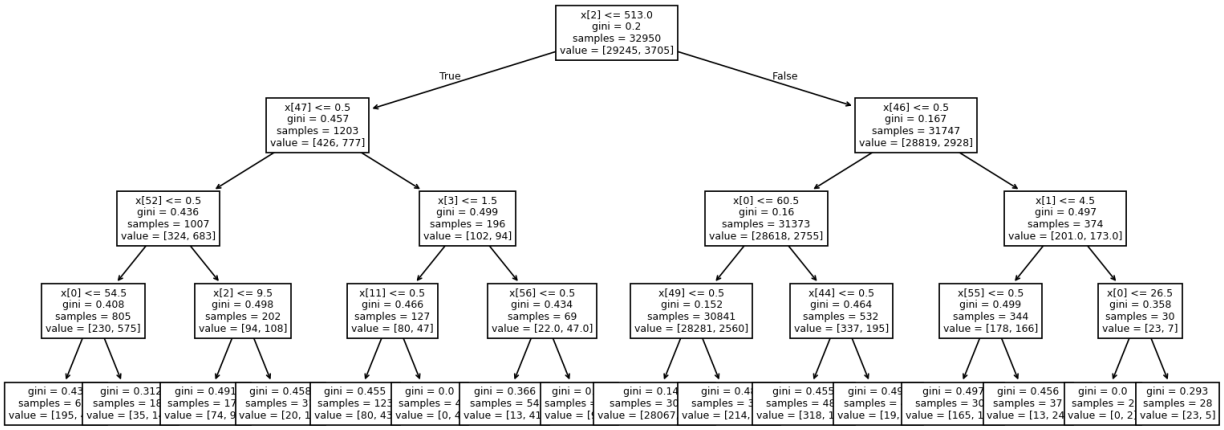
---

4. Decision Tree

```python
#Decision Tree
dtree = DecisionTreeClassifier(max_depth = 4)
dtree.fit(x_train , y_train)
y_train_pred = dtree.predict(x_train)
y_test_pred= dtree.predict(x_test)

#Training
print(confusion_matrix(y_train, y_train_pred))
print(accuracy_score(y_train, y_train_pred))
print(classification_report(y_train, y_train_pred))

#Testing
print(confusion_matrix(y_test, y_test_pred))
print(accuracy_score(y_test, y_test_pred))
print(classification_report(y_test, y_test_pred))
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[[28896   349]
 [ 2938   767]]
0.9002427921092564
              precision    recall  f1-score   support

           0       0.91      0.99      0.95     29245
           1       0.69      0.21      0.32      3705

    accuracy                           0.90     32950
   macro avg       0.80      0.60      0.63     32950
weighted avg       0.88      0.90      0.88     32950

[[7195  108]
 [ 755  180]]
0.895241563486283
              precision    recall  f1-score   support

           0       0.91      0.99      0.94      7303
           1       0.62      0.19      0.29       935

    accuracy                           0.90      8238
   macro avg       0.77      0.59      0.62      8238
weighted avg       0.87      0.90      0.87      8238
```

```python
#Plotting Tree
plt.figure(figsize = (25,6))
tree.plot_tree(dtree)
print(plt.show())
```



5. Model Training

```
parameters = {'criterion' : ['gini' , 'entropy' , 'log_loss'],'splitter' : ['best' , 'random'],'max_depth' : [1,2,3,4,5],'max_features' : ['sqrt' , 'auto' , 'log2']}

dtree2 = DecisionTreeClassifier(random_state = 42)
grid = GridSearchCV(dtree2 , param_grid = parameters , cv = 5 ,scoring = 'accuracy')
grid.fit(x_train , y_train)

print(grid.best_params_)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

{'criterion': 'entropy', 'max_depth': 3, 'max_features': 'log2', 'splitter': 'best'}
```

6. Model Prediction

```
y_train_pred_dt2 = grid.predict(x_train)
y_test_pred_dt2 = grid.predict(x_test)

print(y_train_pred_dt2,  y_test_pred_dt2)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

[0 0 0 ... 1 0 0] [0 0 0 ... 0 0 0]
```

7. Model Evaluation

```
#Training
print(confusion_matrix(y_train, y_train_pred_dt2))
print(accuracy_score(y_train, y_train_pred_dt2))
print(classification_report(y_train, y_train_pred_dt2))

#Testing
print(confusion_matrix(y_test, y_test_pred_dt2))
print(accuracy_score(y_test, y_test_pred_dt2))
print(classification_report(y_test, y_test_pred_dt2))
```

```
[[28865   380]
 [ 2970   735]]
0.8983308042488619
              precision    recall  f1-score   support

           0       0.91      0.99      0.95     29245
           1       0.66      0.20      0.30      3705

    accuracy                           0.90     32950
   macro avg       0.78      0.59      0.63     32950
weighted avg       0.88      0.90      0.87     32950

[[7198  105]
 [ 761  174]]
0.8948773974265598
              precision    recall  f1-score   support

           0       0.90      0.99      0.94      7303
           1       0.62      0.19      0.29       935

    accuracy                           0.89      8238
   macro avg       0.76      0.59      0.61      8238
weighted avg       0.87      0.89      0.87      8238
```

CONCLUSION

This project helped me gain experience in data preprocessing, feature engineering, model training, evaluation, and interpretation.